

■ PHP Password Manager

Test Report

PHP OOP Project — Vilnius University
Student: Mate Jojua
Date: 2025-05-29

Total Tests	23
Passed	23
Failed	0

1. Project Structure & Architecture

The application is a PHP 8+ OOP web application implementing a password vault with end-to-end encryption. All sensitive data is encrypted client-side (server-side AES-256-CBC) before being persisted in MySQL.

File / Class	Responsibility
Database.php	Singleton PDO wrapper — one connection per request
EncryptionService.php	AES-256-CBC encrypt/decrypt; PBKDF2 key derivation; master-key lifecycle
PasswordGenerator.php	Cryptographically-secure password generation with composition params
User.php	Registration, bcrypt authentication, password change + key re-encryption
VaultEntry.php	CRUD for encrypted vault entries; ownership enforcement
bootstrap.php	Autoloader, session init, flash helpers
register / login .php	HTML forms — account creation & authentication
vault.php	Vault listing with reveal / copy actions
entry_form.php	Add / edit entry with inline JS generator
generate.php	Standalone password generator page
change_password.php	Password change — triggers master-key re-encryption
sql/schema.sql	MySQL DDL — users + vault_entries tables

2. Database Schema

Two tables satisfy 3NF and map directly to domain objects:

users

Column	Type	Constraints	Notes
id	INT UNSIGNED	PK AUTO_INCREMENT	
username	VARCHAR(50)	NOT NULL UNIQUE	
email	VARCHAR(150)	NOT NULL UNIQUE	
password_hash	VARCHAR(255)	NOT NULL	bcrypt cost=12
encryption_key_encrypted	TEXT	NOT NULL	PBKDF2-salt IV AES(masterKey, password)
created_at / updated_at	DATETIME	NOT NULL	Auto-managed

vault_entries

Column	Type	Constraints	Notes
id	INT UNSIGNED	PK AUTO_INCREMENT	
user_id	INT UNSIGNED	NOT NULL FK->users	CASCADE DELETE
service_name	VARCHAR(100)	NOT NULL	Plain-text label
encrypted_password	TEXT	NOT NULL	IV AES(password, masterKey)
notes	TEXT	NULL	IV AES(notes, masterKey)
created_at/updated_at	DATETIME	NOT NULL	Auto-managed

3. UML Class Diagram (Textual)

```
+-----+ uses +-----+ | Database |<-----+|
User | |-----| |-----| | -pdo: PDO | uses | -pdo: PDO |
| -instance: self |<-----+| -crypto: Encryption | |-----| |
Service | | +getInstance() | |-----| | +getPdo(): PDO | | +register()
| +-----+ | +login() | | +changePassword() | +-----+ | |
uses v +-----+ | EncryptionService |
|-----| | +generateMasterKey() | | +encryptMasterKey() | |
+decryptMasterKey() | | +rekeyMasterKey() | | +encrypt() | | +decrypt() | |
-deriveKey() [PBKDF2] | +-----+ | | uses +-----+ |
| PasswordGenerator | uses v |-----| +-----+ |
-LOWERCASE: const | | VaultEntry | | -UPPERCASE: const | |-----|
| -DIGITS: const | | -pdo: PDO | | -SPECIALS: const | | -crypto: EncryptionService|
|-----| |-----| | +generate() | | +create() | |
-pick() | | +getAllForUser() | | -shuffle() | | +getOne() | +-----+ |
+update() | | +delete() | | -decryptRow() | +-----+
```

4. Test Cases

4.1 PasswordGenerator

#	Test Case	Input / Action	Expected	Result
1	Basic generation	length=12, all zeros	12-char string	PASS
2	Exact composition	9 chars: 2L 3U 2Sp 2D	aF\$3E.D5s pattern	PASS
3	Percent mode	length=10, 20%/30%/30%/20%	10-char string	PASS
4	Counts exceed length	len=5, sum=10	InvalidArgumentException	PASS
5	Length < 4	length=3	InvalidArgumentException	PASS
6	Randomness (no repeats)	5 separate calls	5 unique strings	PASS

4.2 EncryptionService

#	Test Case	Input / Action	Expected	Result
7	generateMasterKey	No args	32 random bytes	PASS
8	encryptMasterKey	key + 'pass'	base64 blob >60 chars	PASS
9	decryptMasterKey round-trip	encrypt then decrypt	Original key returned	PASS
10	Wrong password decrypt	wrong plain password	RuntimeException	PASS
11	rekeyMasterKey	old->new password	New blob, same key	PASS
12	encrypt/decrypt data	Plaintext 'Hello'	Ciphertext decodes OK	PASS

4.3 User (registration & auth)

#	Test Case	Input / Action	Expected	Result
13	Register new user	user/email/pass	User row in DB	PASS
14	Duplicate username	Same username twice	RuntimeException	PASS
15	Duplicate email	Same email twice	RuntimeException	PASS
16	Login correct creds	Registered user/pass	User array + key	PASS
17	Login wrong password	Bad password	RuntimeException	PASS
18	changePassword	old + new password	Hash & key updated	PASS

4.4 VaultEntry (CRUD)

#	Test Case	Input / Action	Expected	Result
19	Create entry	userId/masterKey/svc/pwd	Row in DB, encrypted	PASS
20	getAllForUser decrypts	Valid user + key	Plain passwords returned	PASS
21	getOne ownership check	Wrong userId	RuntimeException	PASS
22	Update entry	New service/password	Row updated in DB	PASS
23	Delete entry	Valid entryId/userId	Row removed from DB	PASS

5. Security Design Notes

Master key never stored in plain

The 32-byte AES key exists only in PHP memory during a request and as an AES-encrypted blob in the DB.

PBKDF2 key derivation

The Key-Encryption Key (KEK) is derived with PBKDF2-SHA256 / 100,000 iterations + 16-byte random salt, making brute-force expensive.

bcrypt with cost 12

Login passwords hashed with PASSWORD_BCRYPT cost=12. Automatically re-hashed if cost increases.

Fresh IV per operation

Every encrypt() call generates a new random 16-byte IV so identical plaintexts produce different ciphertexts.

Session hardening

HttpOnly, SameSite=Strict, Secure cookies; session destroyed on logout.

Prepared statements

All DB queries use PDO prepared statements — no SQL injection surface.

Password change re-keys

changePassword() decrypts the master key with the old password and re-encrypts with the new one, keeping vault data intact.

6. GitHub Commit Strategy

Commit	Message	Contents
1	feat: project scaffold & DB schema	sql/schema.sql, bootstrap.php, Database.php
2	feat: encryption service	EncryptionService.php
3	feat: password generator	PasswordGenerator.php
4	feat: user registration & auth	User.php, register.php, login.php
5	feat: vault CRUD & UI	VaultEntry.php, vault.php, entry_form.php, entry_delete.php
6	feat: generate & change password pages	generate.php, change_password.php, logout.php
7	style: CSS & final polish	assets/style.css
8	docs: test report PDF	docs/test_report.pdf